

MEDIATEK

MT8676 Hypervisor T-Box User Manual

Version: 1.1
Release date: 2024-12-06

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-11-13	Yali Wu	Official release
1.1	2024-12-06	Yali Wu	Modified Figure 1-1. Hypervisor T-Box architecture

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 T-Box..... 4

1.1 Overview 4

1.1.1 Brief Introduction 4

1.1.2 T-Box Abbreviations 4

1.2 Architecture/Process Overview..... 4

1.2.1 T-Box Architecture..... 4

1.2.2 T-Box API Usage..... 5

1.3 Frequently Asked Questions/Troubleshooting 27

1.3.1 SIM/CALL/SMS/Telephony Network Tips 27

1.3.2 Data Tips 28

1.3.3 Network Tips 28

1.3.4 IMS Tips..... 28

Exhibit 1 Terms and Conditions 29

List of Figures

Figure 1-1. Hypervisor T-Box architecture..... 4

List of Tables

Table 1-1. Abbreviations..... 4

Table 1-2. Modem state and IMEI interface description 5

Table 1-3. SIM interface description 5

Table 1-4. Telephony NW interface description 9

Table 1-5. Data interface description 13

Table 1-6. Call module interface description..... 20

Table 1-7. Call interface description 22

Table 1-8. IMS interface description 23

Table 1-9. eCall control interface description..... 24

Table 1-10. AT blacklist interface description 26

Table 1-11. MIPC keep alive interface description 27

1 T-Box

1.1 Overview

1.1.1 Brief Introduction

This section introduces the Hypervisor T-Box architecture, API usage and precautions.

1.1.2 T-Box Abbreviations

Table 1-1. Abbreviations

Abbreviation	Explanation
API	Application Programming Interface
APN	Access Point Name
T-Box	Telematics-BOX

1.2 Architecture/Process Overview

1.2.1 T-Box Architecture

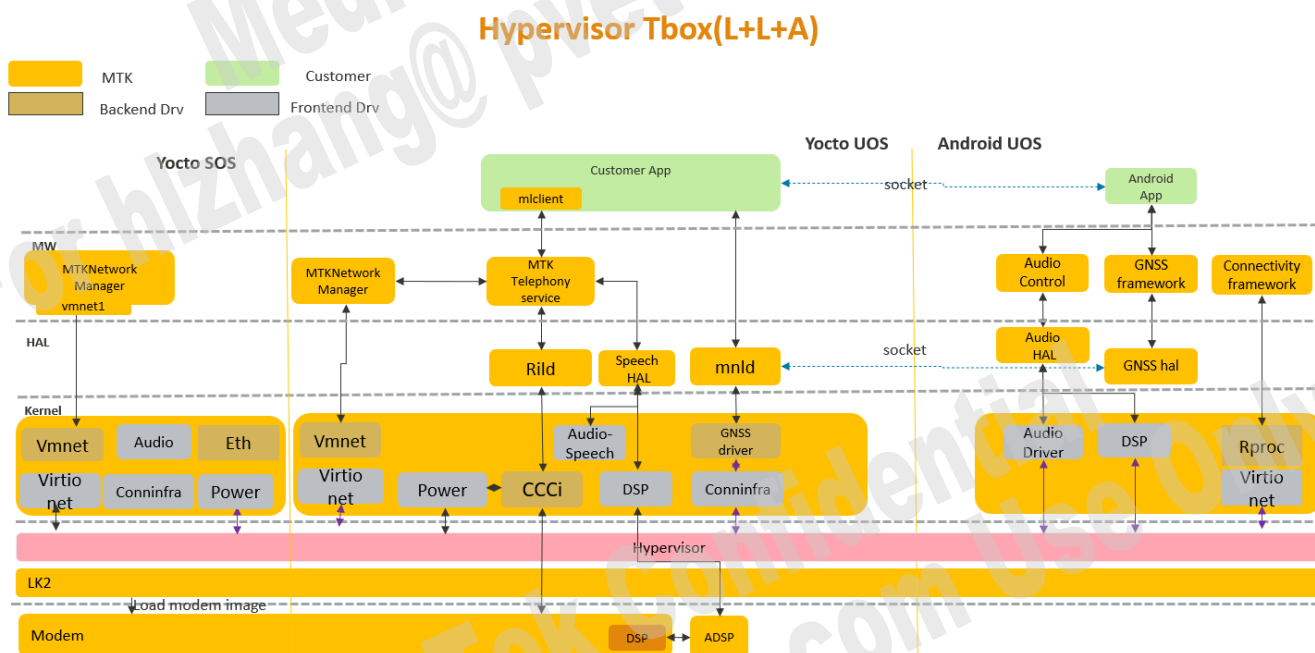


Figure 1-1. Hypervisor T-Box architecture

1.2.2 T-Box API Usage

1.2.2.1 Modem State/IMEI Interface and Call Sequence Description

All other T-Box APIs require modem ready to work properly except ML_GetModemStat.

Table 1-2. Modem state and IMEI interface description

Interface/Struct	Description
Int ML_GetModemStat(char *stat, uint32_t statlen)	Get modem state, stat will return "ready" if modem has boot up success.
Int ML_GetImei(char* imei, size_t imeiLen)	Get IMEI of device. On success, 0 is returned. On error, -1 is returned.

1.2.2.2 SIM Interface and Call Sequence Description

All other SIM APIs require SIM present to work properly except ML_GetCardStatus.

Table 1-3. SIM interface description

Interface/Struct	Description
<pre>typedef enum { E_ML_SIM_CARD_STATE_UNKNOWN = 0xB01, /**< Card state unknown. */ E_ML_SIM_CARD_STATE_ABSENT = 0xB02, /**< Card is absent. */ E_ML_SIM_CARD_STATE_PRESENT = 0xB03, /**< Card is present. */ E_ML_SIM_CARD_STATE_ERROR_UNKNOWN = 0xB04, /**< Unknown error state. */ E_ML_SIM_CARD_STATE_ERROR_POWER_DOWN = 0xB05, /**< Power down. */ E_ML_SIM_CARD_STATE_ERROR_POLL_ERROR = 0xB06, /**< Poll error. */ E_ML_SIM_CARD_STATE_ERROR_NO_ATR_RECEIVED = 0xB07, /**< Failed to receive an answer to reset. */ E_ML_SIM_CARD_STATE_ERROR_VOLT_MISMATCH = 0xB08, /**< Voltage mismatch. */ E_ML_SIM_CARD_STATE_ERROR_PARITY_ERROR = 0xB09, /**< Parity error. */ E_ML_SIM_CARD_STATE_ERROR_SIM_TECHNICAL_PROBLEMS= 0xB0A, /**< Card returned technical problems. */ }E_ML_SIM_CARD_STATE_TYPE_T; /**< Card state. */</pre>	SIM card status structure. Mainly focus on PRESENT, which means the card has been recognized, and ABSENT, which means the SIM card has not been recognized.

Interface/Struct	Description
Int32_t ML_Sim_GetCardStatus(ML_SIM_CARD_STATUS_INFO_T *pvsSimStatus)	Get SIM state. The value of pvsSimStatus will return "present" when SIM card is present. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetICCID(char *iccid, size_t iccidLen)	Get the SIM card ICCID. ICCID (Integrated Circuit Card Identifier) is a number used to uniquely identify each SIM card. int return value: On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetMsisdn(char *msisdn, size_t msisdnLen)	Get the SIM card Msisdn. Here it refers to the phone number. int return value: On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetImsi(char *imsi, size_t imsilen)	Get the SIM card IMSI. IMSI (International Mobile Subscriber Identity) is a number used to uniquely identify a user in a mobile network. int return value: On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetMccMnc(char *mcc, size_t mccLen, char*mnc, size_t mncLen)	Get SIM card mccmnc. MCCMNC (Mobile Country Code and Mobile Network Code) is a number used to uniquely identify a mobile network operator. MCCMNC consists of two parts: MCC (Mobile Country Code) and MNC (Mobile Network Code). int return value: On success, 0 is returned. On error, -1 is returned.
int32_t ML_Sim_GetCardFullStatus (ML_SIM_CARD_FULL_STATUS_INFO_T * pvsSimStatus)	Get complete SIM card information. The returned structure contains card status, pin status, SIM app information, etc. int return value: On success, 0 is returned. On error, -1 is returned.
typedef struct { E_ML_CardState card_state; E_ML_PinState universal_pin_state; /* applicable to USIM and CSIM: E_ML_PINSTATE_xxx */ int gsm_umts_subscription_app_index; /* value < E_ML_CARD_MAX_APPS, -1 if none */ int cdma_subscription_app_index; /* value < E_ML_CARD_MAX_APPS, -1 if none */ int ims_subscription_app_index; /* value < E_ML_CARD_MAX_APPS, -1 if none */ int num_applications; /* value <= E_ML_CARD_MAX_APPS E_ML_AppStatus applications[E_ML_CARD_MAX_APPS]; } ML_SIM_CARD_FULL_STATUS_INFO_T;	SIM card detailed information structure.
int ML_SIM_ChangePin (int32_t slotIndex,	PIN1: Used to protect the SIM card from

Interface/Struct	Description
<pre>const char * oldPin, const char * newPin, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	unauthorized use. PIN2: Used to protect specific SIM card functions, such as fixed dialing numbers. PUK: Used to unlock a SIM card locked by PIN1. PUK2: Used to unlock functions locked by PIN2. Modify the pin value. Note: You need to lock the card before changing the pin. int return value: On success, 0 is returned. On error, -1 is returned. Output parameter: errorCode; On error, errorCode is returned. For errorCode, refer to the RIL_Errno enumeration in ril.h. Output parameter: retryTimes; The number of attempts, the default is 3, and -1 is returned when the pin input is wrong.
<pre>int ML_SIM_ChangePin2 (int32_t slotIndex, const char * oldPin2, const char * newPin2, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	Modify the value of pin2. Note: You need to lock before changing pin2. Int return value: On success, 0 is returned. On error, -1 is returned. Output parameter: errorCode; On error, errorCode is returned. For errorCode, refer to the RIL_Errno enumeration in ril.h. Output parameter: retryTimes; The number of attempts, the default is 3, and -1 is returned when the pin input is wrong.
<pre>int ML_SIM_EnterPin (int32_t slotIndex, const char * pin, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	Unlock the pin. Note: The pin can only be unlocked after it is locked. Int return value: On success, 0 is returned. On error, -1 is returned. Output parameter: errorCode; On error, errorCode is returned. For errorCode, refer to the RIL_Errno enumeration in ril.h. Output parameter: retryTimes; The number of attempts, the default is 3, and -1 is returned when the pin input is wrong.
<pre>int ML_SIM_EnterPin2 (int32_t slotIndex, const char * pin2, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	Unlock pin2. Note: Pin2 can only be unlocked after locking int return value: On success, 0 is returned. On error, -1 is returned. Output parameter: errorCode; On error, errorCode is returned. For errorCode, refer to the RIL_Errno enumeration in ril.h.

Interface/Struct	Description
	Output parameter: retryTimes; The number of attempts, the default is 3, and -1 is returned when pin2 input is wrong.
<pre>int ML_SIM_EnterPuk (int32_t slotIndex, const char * puk, const char * pin, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	Unlock PUK. Used to unlock a SIM card locked by PIN1. When the user enters the wrong PIN1 for more than a certain number of times, the SIM card will be locked. At this time, you need to enter the PUK code to unlock and reset PIN1. Note: You need to lock before you can unlock puk int return value: On success, 0 is returned. On error, -1 is returned. Output parameter: errorCode; On error, errorCode is return. For errorCode, refer to the RIL_Errno enumeration in ril.h. Output parameter: retryTimes; The number of attempts, the default is 3, and -1 is returned when puk is entered incorrectly.
<pre>int ML_SIM_EnterPuk2 (int32_t slotIndex, const char * puk2, const char * pin2, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	Unlock PUK2. Used to unlock a SIM card locked by PIN2. When the user enters the wrong PIN2 for more than a certain number of times, the SIM card will be locked. At this time, you need to enter the PUK code to unlock and reset PIN2. Note: You need to lock before you can unlock puk int return value: On success, 0 is returned. On error, -1 is returned. Output parameter: errorCode; On error, errorCode is return. For errorCode, refer to the RIL_Errno enumeration in ril.h. Output parameter: retryTimes; The number of attempts, the default is 3, and -1 is returned when puk2 is entered incorrectly.
<pre>int ML_SIM_SetFacilityLock (int32_t slotIndex, const char * facility, int32_t lockState, const char * password, int32_t serviceClass, const char * appld, int32_t * errorCode, int32_t * retryTimes)</pre>	Set the pin lock state. Note: This can only be set when the device is unlocked. If this function is called when the device is locked, an error will be returned. Input parameters: Facility: device type LockState: 1: locked 0: unlocked Int return value: On success, 0 is returned. On error, -1 is returned.
<pre>int ML_SIM_IccCloseLogicalChannelBySlot (int32_t slotIndex, int32_t channel))</pre>	Close the IO logical channel. int return value: On success, 0 is returned. On error, -1 is returned.
<pre>int ML_SIM_IccOpenLogicalChannelBySlot (int32_t slotIndex, const char * aid, int32_t p2, int32_t channel)</pre>	Open IO logical channel. int return value: On success, 0 is returned. On error, -1 is returned.

Interface/Struct	Description
<pre>int ML_SIM_IccTransmitApduLogicalChannelBySlot (int32_t slotIndex, int32_t channel, int32_t cla, int32_t instruction, int32_t p1, int32_t p2, int32_t p3, const char * data)</pre>	Send apdu command. Input parameters: int32_t channel : logical channel int32_t cla: Class byte of APDU command int32_t instruction: Instruction byte of APDU command. int32_t p1, int32_t p2, int32_t p3, Refers to the parameters of APDU command. Used to further specify the operation of the command const char * data Data field of APDU command int return value: On success, 0 is returned. On error, -1 is returned.
<pre>int ML_SIM_SetCardPower (int32_t CardPowerState)</pre>	Set the SIM card power state, that is, reset the SIM card. Input parameters: int32_t CardPowerState 0: power off 1: power on 2: pass through mode int return value: On success, 0 is returned. On error, -1 is returned.

1.2.2.3 Telephony NW Interface and Call Sequence Description

Table 1-4. Telephony NW interface description

Interface/Struct	Description
<pre>typedef enum { E ML NW PS UNKNOWN = 0x00, E ML NW PS ATTACHED = 0x01, E ML NW PS DETACHED = 0x02, }E ML NW PS REG_STATE_TYPE_T;</pre>	Struct of PS registration state.
<pre>typedef enum { E ML NW CS UNKNOWN = 0x00, E ML NW CS ATTACHED = 0x01, E ML NW CS DETACHED = 0x02, }E ML NW CS REG_STATE_TYPE_T;</pre>	Struct of CS registration state.
<pre>int ML_GetNetState(E ML NW PS REG_STATE_TYPE_T * ps, E ML NW CS REG_STATE_TYPE_T * cs)</pre>	Get CS/PS registration state. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.
<pre>int32_t ML_GetOperatorCode (</pre>	Get operator name. Value of op_code is "1" when sim operator is

Interface/Struct	Description
uint8_t * op_code)	CU. Value of op_code is "0" when SIM operator is CMCC. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.
typedef enum { E_ML_NW_RAT_GSM = 0, E_ML_NW_RAT_UTRAN = 2, E_ML_NW_RAT_GSMW_EGPRS = 3, E_ML_NW_RAT_UTRANW_HSDPA = 4, E_ML_NW_RAT_UTRANW_HSUPA = 5, E_ML_NW_RAT_UTRANW_HSDPA_AND_HSUPA = 6, E_ML_NW_RAT_E_UTRAN = 7, #ifdef SPM_TELEPHONY_NR_SUPPORT E_ML_NW_RAT_NR = 8, #endif }E_ML_NW_RADIO_ACCESS_TYPE_T;	Struct of radio access type.
int ML_GetRadioAccessType (E_ML_NW_RADIO_ACCESS_TYPE_T * access_type)	Get radio access type of network registered. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.
int ML_GetSignalStrength (int8_t * sig_level)	Get signal strength level. Sig_level: 0~4. For signal level thresholds, refer to AOSP default classification. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.
int ML_SetAirplaneMode (uint8_t on_off)	Set airplane mode. Input value "on_off" will be set to 1 if airplane on. Input value "on_off" will be set to 0 if airplane off. Function return value is int type. On success, 0 is returned. On error, -1 is returned. Notice: This interface should be used after SIM present.
typedef void (*ml_signal_strength_cb_t)(ML_SignalStrength *state);	Signal strength call back function.
int ML_SignalStreghthInit (ml_signal_strength_cb_t evt_cb)	Init signal strength call back function. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
typedef enum { E_ML_AUTO = 0, E_ML_2GONLY = 1, E_ML_3GONLY = 2, E_ML_4GONLY = 3, #ifdef SPM_TELEPHONY_NR_SUPPORT	Struct of network mode.

Interface/Struct	Description
<pre> E_ML_5GONLY = 4, E_ML_5GAUTO = 5, E_ML_5G4GAUTO = 6, #endif }ml_nw_net_mode_e; </pre>	
<pre> int ML_SetNetMode (ml_nw_net_mode_e mode) </pre>	Set network mode. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
<pre> int ML_GetNetMode (ml_nw_net_mode_e * mode) </pre>	Get network mode. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
<pre> int ML_GetFullSignalStrength (int slot_id, ML_SignalStrength * fullMsg) </pre>	Get all signal strength information. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
<pre> int getAvailableNetworksWithActAsync (int slot_id) </pre>	Scans for available networks with ACT Response function is ML_InitAvailableNetworksWithActResponseCb() Function return value is int type. On success, 0 is returned. On error, -1 is returned.
<pre> int getCellInfoListAsync (int slot_id) </pre>	Request all of the current cell information known to the radio. The radio must return a list of all current cells, including the neighboring cells. If for a particular cell information isn't known then the appropriate unknown value will be returned. Response function is ML_getCellInfoListResponseCb() Function return value is int type. On success, 0 is returned. On error, -1 is returned.
<pre> int ML_getCellInfoListResponseCb (int32_t slot_id, ml_get_cell_info_list_response response) </pre>	Response ML_getCellInfoListResponseCb of getCellInfoListAsync
<pre> int32_t getDataRegistrationState (int32_t slot_id) </pre>	Request current data registration state. Response function is ML_getDataRegistrationStateResponseCb() Function return value is int type. On success, 0 is returned. On error, -1 is returned.
<pre> int ML_getDataRegistrationStateResponseCb (int32_t slot_id, ml_get_data_registration_state_response response) </pre>	Response getDataRegistrationStateResponseCb of getDataRegistrationState
<pre> int32_t getNetworkSelectionMode (int32_t slot_id, ML_NetworkSelectionMode * mode) </pre>	Query current network selection mode. Function return value is int type. On success, 0 is returned. On error, -1 is returned.

Interface/Struct	Description
int32_t getNitzTime (int32_t slot_id, char ** nitzTime, int32_t tz_valid)	acquire NITZ information (UTC time and time zone). need free nitzTime after used asprintf(&nitzTime, "%02d/%02d/%02d,%02d:%02d:%02d%+03d,%d", pNitzInfo->year % 100, pNitzInfo->month, pNitzInfo->day, pNitzInfo->hour, pNitzInfo->minute, pNitzInfo->second, pNitzInfo->time_zone_offset_minutes / 15, pNitzInfo->daylight_saving_offset_minutes / 60); Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int getOperatorSync (int slot_id, char ** longName, char ** shortName, char ** numeric)	Request current operator ONS or EONS. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int setUsageSetting (int slot_id, int usageSetting)	Set the UE usage setting for data/voice centric usage. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int getUsageSetting (int slot_id, int * usageSetting)	Get the UE usage setting for data/voice centric usage. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int32_t getVoiceRegistrationState (int32_t slot_id)	Request current voice registration state. Response function is ML_getVoiceRegistrationStateResponseCb() Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_getVoiceRegistrationStateResponseCb (int32_t slot_id, ml_get_voice_registration_state_response response)	Response function of getVoiceRegistrationState
int ML_CsNetworkInit (int32_t slot_id, ml_cs_network_change_cb_t evt_cb)	Initialization network module, and callback function registered. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_PsNetworkInit (int32_t slot_id, ml_ps_network_change_cb_t evt_cb)	Initialization ps network module, and callback function registered. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_imsNetworkStateChangedInit (int32_t slot_id, ml_ims_network_state_changed_cb_t evt_cb)	Indicates when IMS registration state has changed. To get IMS registration state and IMS SMS format, callee needs to invoke getImsRegState(). Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_networkStateChangedInit (int32_t slot_id, ml_network_state_changed_cb_t evt_cb)	Indicates when voice or data network state changed. Callee must invoke getOperator(),getVoiceRegistrationState(),getDataRegistrationState()

Interface/Struct	Description
	Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_nitzTimeReceivedInit (int32_t slot_id, ml_nitz_time_received_t evt_cb)	Initialization nitz time change, and callback function registered. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int32_t setNetworkSelectionModeAutomatic (int32_t slot_id)	Specify that the network must be selected automatically. Response function is ML_setNetworkSelectionModeAutomaticResponseCb() Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_setNetworkSelectionModeAutomaticResponseCb (int32_t slot_id, ml_set_network_selection_mode_automatic_response response)	Response function of setNetworkSelectionModeAutomatic
int32_t setNetworkSelectionModeManual (int32_t slot_id, const char * operatorNumeric, ML_AccessNetwork ran)	Manually select a specified network. This request must not respond until the new operator is selected and registered. Per TS 23.122, the RAN is just the initial suggested value. If registration fails, the RAN is not available afterwards, or the RAN is not within the network types specified by IRadioNetwork::setAllowedNetworkTypeBitmap, then the modem will need to select the next best RAN for network registration. Response function is ML_setNetworkSelectionModeManualResponseCb() Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_setNetworkSelectionModeManualResponseCb (int32_t slot_id, ml_set_network_selection_mode_manual_response response)	Response function of setNetworkSelectionModeManual

1.2.2.4 Data Interface and Call Sequence Description

Table 1-5. Data interface description

Interface/Struct	Description
typedef enum { ML_DATA_CALL_TYPE_IPV4 = 0, ML_DATA_CALL_TYPE_IPV6, ML_DATA_CALL_TYPE_IPV4V6, } ml_data_call_ip_family_e;	IP family enum defined.

Interface/Struct	Description
<pre>typedef enum { ML_APN_PDP_TYPE_IPV4 = 0, ML_APN_PDP_TYPE_PPP, ML_APN_PDP_TYPE_IPV6, ML_APN_PDP_TYPE_IPV4V6, } ml_apn_pdp_type_e;</pre>	APN PDP type enum defined.
<pre>typedef enum { ML_APN_AUTH_PROTO_DEFAULT = 0, ML_APN_AUTH_PROTO_NONE, ML_APN_AUTH_PROTO_PAP, ML_APN_AUTH_PROTO_CHAP, ML_APN_AUTH_PROTO_PAP_CHAP, } ml_apn_auth_proto_e;</pre>	APN authentication protocol type enum defined.
<pre>typedef struct { unsigned char profile_idx; ml_apn_pdp_type_e pdp_type; ml_apn_auth_proto_e auth_proto; char apn_name[ML_APN_NAME_SIZE]; char username[ML_APN_USERNAME_SIZE]; char password[ML_APN_PASSWORD_SIZE]; } ml_apn_info_s</pre>	Struct of apn info profile_idx: 1 – 8 (1: Public profile ID; 2-8: Private profile ID, support up to 7 private APNs) pdp_type: IPV4/IPV6/IPV4V6; for details, refer to ml_apn_pdp_type_e enum auth_proto: default/none/pap/chap/pap_chap apn_name: APN name
<pre>typedef enum { ML_DATA_CALL_ERROR_NONE = 0, ML_DATA_CALL_ERROR_INVALID_PARAMS, } ml_data_call_error_e;</pre>	Data call error enum defined.
<pre>typedef enum { ML_DATA_CALL_IDLE, ML_DATA_CALL_CONNECTING, ML_DATA_CALL_CONNECTED, ML_DATA_CALL_DISCONNECTING, ML_DATA_CALL_DISCONNECTED, ML_DATA_CALL_RETRYING, ML_DATA_CALL_FAILED, ML_DATA_CALL_SCANNING, } ml_data_call_state_e;</pre>	Data call state enum defined.
<pre>struct ml_v4_address_status { struct in_addr ip; struct in_addr gateway; struct in_addr pri_dns; struct in_addr sec_dns; };</pre>	IPV4 address status struct.
<pre>struct ml_v6_address_status { struct in6_addr ip; struct in6_addr gateway; struct in6_addr pri_dns; struct in6_addr sec_dns; };</pre>	IPV6 address status struct.

Interface/Struct	Description
<pre>struct ml_pkt_stats { unsigned long pkts_tx; unsigned long pkts_rx; long long bytes_tx; long long bytes_rx; unsigned long pkts_dropped_tx; unsigned long pkts_dropped_rx; };</pre>	Packet status struct.
<pre>struct ml_v4_info { char name[16]; ml_data_call_state_e state; bool reconnect; struct ml_v4_address_status addr; struct ml_pkt_stats stats; };</pre>	IPV4 info struct. name: APN name state: IPV4 data call state reconnect: IPV4 re-dial flag addr: IPV4 address stats: IPV4 packet status
<pre>struct ml_v6_info { char name[16]; ml_data_call_state_e state; bool reconnect; struct ml_v6_address_status addr; struct ml_pkt_stats stats; };</pre>	IPV6 info struct. name: APN name state: IPV6 data call state reconnect: IPV6 re-dial flag addr: IPV6 address stats: IPV6 packet status
<pre>typedef struct { char profile_idx; ml_data_call_ip_family_e ip_family; struct ml_v4_info v4; struct ml_v6_info v6; } ml_data_call_info_s;</pre>	Data call info struct.
<pre>typedef struct { char profile_idx; char name[16]; ml_data_call_ip_family_e ip_family; ml_data_call_state_e state; ml_data_call_error_e err; struct ml_v4_address_status v4; struct ml_v6_address_status v6; } ml_data_call_state_s;</pre>	Data call state struct.
<pre>typedef void (*ml_data_call_evt_cb_t)(ml_data_call_state_s *state);</pre>	Data callback function. Parameters: in: ml_data_call_state_s *state
<pre>typedef enum { PDP_FAIL_NONE = 0, /* an integer cause code defined in TS 24.008 section 6.1.3.1.3 or TS 24.301 Release 8+ Annex B. If the implementation does not have access to the exact cause codes, then it should return one of the following values, as the UI layer needs to distinguish these cases for error notification and potential retries. */ PDP_FAIL_OPERATOR_BARRED = 0x08,</pre>	Data call fail cause enum defined.

Interface/Struct	Description
PDP_FAIL_NAS_SIGNALLING = 0x0E, PDP_FAIL_MBMS_CAPABILITIES_INSUFFICIENT = 0x18, PDP_FAIL_LLC_SNDP = 0x19, PDP_FAIL_INSUFFICIENT_RESOURCES = 0x1A, PDP_FAIL_MISSING_UNKNOWN_APN = 0x1B, PDP_FAIL_UNKNOWN_PDP_ADDRESS_TYPE = 0x1C, PDP_FAIL_USER_AUTHENTICATION = 0x1D, PDP_FAIL_ACTIVATION_REJECT_GGSN = 0x1E, PDP_FAIL_ACTIVATION_REJECT_UNSPECIFIED = 0x1F, PDP_FAIL_SERVICE_OPTION_NOT_SUPPORTED = 0x20, PDP_FAIL_SERVICE_OPTION_NOT_SUBSCRIBED = 0x21, PDP_FAIL_SERVICE_OPTION_OUT_OF_ORDER = 0x22, PDP_FAIL_NSAPI_IN_USE = 0x23, PDP_FAIL_REGULAR_DEACTIVATION = 0x24, PDP_FAIL_QOS_NOT_ACCEPTED = 0x25, PDP_FAIL_NETWORK_FAILURE = 0x26, PDP_FAIL_UMTS_REACTIVATION_REQ = 0x27, PDP_FAIL_FEATURE_NOT_SUPP = 0x28, PDP_FAIL_TFT_SEMANTIC_ERROR = 0x29, PDP_FAIL_TFT_SYNTAX_ERROR = 0x2A, PDP_FAIL_UNKNOWN_PDP_CONTEXT = 0x2B, PDP_FAIL_FILTER_SEMANTIC_ERROR = 0x2C, PDP_FAIL_FILTER_SYNTAX_ERROR = 0x2D, PDP_FAIL_PDP_WITHOUT_ACTIVE_TFT = 0x2E, PDP_FAIL_MULTICAST_GROUP_MEMBERSHIP_TIMEOUT = 0x2F, PDP_FAIL_BCM_VIOLATION = 0x30, PDP_FAIL_LAST_PDN_DISC_NOT_ALLOWED = 0x31, PDP_FAIL_ONLY_IPV4_ALLOWED = 0x32, PDP_FAIL_ONLY_IPV6_ALLOWED = 0x33, PDP_FAIL_ONLY_SINGLE_BEARER_ALLOWED = 0x34, PDP_FAIL_ESM_INFO_NOT_RECEIVED = 0x35, PDP_FAIL_PDN_CONN_DOES_NOT_EXIST = 0x36, PDP_FAIL_MULTI_CONN_TO_SAME_PDN_NOT_ALLOWED = 0x37, PDP_FAIL_COLLISION_WITH_NW_INITIATED_REQUEST = 0x38, PDP_FAIL_ESM_UNSUPPORTED_QCI_VALUE = 0x3B, PDP_FAIL_BEARER_HANDLING_NOT_SUPPORT = 0x3C, PDP_FAIL_MAX_ACTIVE_PDP_CONTEXT_REACHED = 0x41, PDP_FAIL_UNSUPPORTED_APN_IN_CURRENT_PLMN = 0x42, PDP_FAIL_INVALID_TRANSACTION_ID = 0x51, PDP_FAIL_MESSAGE_INCORRECT_SEMANTIC = 0x5F, PDP_FAIL_INVALID_MANDATORY_INFO = 0x60, PDP_FAIL_MESSAGE_TYPE_UNSUPPORTED = 0x61, PDP_FAIL_MSG_TYPE_NONCOMPATIBLE_STATE = 0x62, PDP_FAIL_UNKNOWN_INFO_ELEMENT = 0x63, PDP_FAIL_CONDITIONAL_IE_ERROR = 0x64, PDP_FAIL_MSG_AND_PROTOCOL_STATE_UNCOMPATIBLE = 0x65, PDP_FAIL_PROTOCOL_ERRORS = 0x6F, /* no retry */ PDP_FAIL_APN_TYPE_CONFLICT = 0x70, PDP_FAIL_INVALID_PCSCF_ADDR = 0x71,	

Interface/Struct	Description
PDP_FAIL_INTERNAL_CALL_PREEMPT_BY_HIGH_PRIO_APN = 0x72, PDP_FAIL_EMM_ACCESS_BARRED = 0x73, PDP_FAIL_EMERGENCY_IFACE_ONLY = 0x74, PDP_FAIL_IFACE_MISMATCH = 0x75, PDP_FAIL_COMPANION_IFACE_IN_USE = 0x76, PDP_FAIL_IP_ADDRESS_MISMATCH = 0x77, PDP_FAIL_IFACE_AND_POL_FAMILY_MISMATCH = 0x78, PDP_FAIL_EMM_ACCESS_BARRED_INFINITE_RETRY = 0x79, PDP_FAIL_AUTH_FAILURE_ON_EMERGENCY_CALL = 0x7A, PDP_FAIL_LOCAL_REJECT_ACT_REQ_DUE_TO_REACH_RETRY_COUNTER = 0x0E0F, PDP_FAIL_TCM_ESM_TIMER_TIMEOUT = 0x0F46, PDP_FAIL_PAM_ATT_PDN_ACCESS_REJECT_IMS_PDN_BLOCK_TEMPL = 0x1402, PDP_FAIL_DATA_NOT_ALLOW = 0x1671, PDP_FAIL_OEM_DCFailureCause_1 = 0x1001, PDP_FAIL_OEM_DCFailureCause_2 = 0x1002, PDP_FAIL_OEM_DCFailureCause_3 = 0x1003, PDP_FAIL_OEM_DCFailureCause_4 = 0x1004, PDP_FAIL_OEM_DCFailureCause_5 = 0x1005, PDP_FAIL_OEM_DCFailureCause_6 = 0x1006, PDP_FAIL_OEM_DCFailureCause_7 = 0x1007, PDP_FAIL_OEM_DCFailureCause_8 = 0x1008, PDP_FAIL_OEM_DCFailureCause_9 = 0x1009, PDP_FAIL_OEM_DCFailureCause_10 = 0x100A, PDP_FAIL_OEM_DCFailureCause_11 = 0x100B, PDP_FAIL_OEM_DCFailureCause_12 = 0x100C, PDP_FAIL_OEM_DCFailureCause_13 = 0x100D, PDP_FAIL_OEM_DCFailureCause_14 = 0x100E, PDP_FAIL_OEM_DCFailureCause_15 = 0x100F, PDP_FAIL_VOICE_REGISTRATION_FAIL = -1, PDP_FAIL_DATA_REGISTRATION_FAIL = -2, PDP_FAIL_SIGNAL_LOST = -3, PDP_FAIL_PREF_RADIO_TECH_CHANGED = -4, PDP_FAIL_RADIO_POWER_OFF = -5, PDP_FAIL_TETHERED_CALL_ACTIVE = -6, PDP_FAIL_ROUTER_ADVERTISEMENT_FAIL = -7, PDP_FAIL_LOST_CONNECTION = 0x10004, PDP_FAIL_ERROR_UNSPECIFIED = 0xffff, }ML_DataCallFailCause;	
<pre>int ML_APN_Get (unsigned char profile_idx, ml_apn_info_s * apn)</pre>	Get APN info from input value "profile_idx". Parameters: in: unsigned char profile_idx out: ml_apn_info_s *apn APN initial flow:

Interface/Struct	Description
	After the card is recognized at startup, mtktelephony service will determine whether /data/vendor/telephony/apn.db exists. If not, it generates /data/vendor/telephony/apn.db according to /vendor/etc/apns_conf.xml. Then it obtains the APN of the SIM card from apn.db through the card's mccmnc and sets the APN to the modem. Notice: This interface needs to be called after the card is recognized, that is, after ML_Sim_GetCardStatus() returns present.
<pre>int ML_APN_Set (const ml_apn_info_s * apn)</pre>	Set APN to modem and apn database(/data/vendor/telephony/apn.db) Parameters: in: const ml_apn_info_s * apn Notice: This interface needs to be called after the card is recognized
<pre>int ML_DataCallInit (ml_data_call_evt_cb_t evt_cb)</pre>	Register the data call status change callback function, and call back the evt_cb function when the data call status changes. Parameters: in: ml_data_call_evt_cb_t evt_cb
<pre>int ML_DataCallStart (const ml_data_call_s * data_call, ml_data_call_error_e * err)</pre>	Establish public/private PDN connection. This is synchronous interface. It sends PDN establishment command to modem, and this interface returns after modem returns. Parameters: in: const ml_data_call_s * data_call out: ml_data_call_error_e * err
<pre>int ML_DataCallStop (char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_error_e * err)</pre>	Disconnect the public/private PDN connection. This is synchronous interface. It sends the PDN establishment command to the modem, and this interface will return only after the modem returns. Parameters: in: char profile_idx in: ml_data_call_ip_family_e ip_family out: ml_data_call_error_e * err
<pre>int ML_DataCallStart_Ext (const ml_data_call_s * data_call, ml_data_call_error_e * err)</pre>	Establish public/private PDN connection.

Interface/Struct	Description
	This asynchronous interface, returns directly after issuing the PDN establishment command, without waiting for the modem execution result. The status of establishing PDN is called back to the APP through the callback function registered by ML_DataCallInit. Parameters: in: const ml_data_call_s * data_call out: ml_data_call_error_e * err
<pre>int ML_DataCallStop_Ext (char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_error_e * err)</pre>	Disconnect public/private PDN connection Asynchronous interface, returns directly after issuing the disconnect PDN command, without waiting for the modem execution result. The disconnect PDN status is called back to the APP through the callback function registered by ML_DataCallInit. Parameters: in: char profile_idx in: ml_data_call_ip_family_e ip_family out: ml_data_call_error_e * err
<pre>int ML_getDataCallReason (int32_t profile_idx, ML_DataCallFailCause * reason)</pre>	When PDN establishment fails, get failcause. Parameters: in: profile_idx in/out: reason
<pre>Int ML_DataCallInfoGet(char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_info_s *info, ml_data_call_error_e *err)</pre>	Get data call information according to profile_idx and ip_family. Parameters: in: char profile_idx in: ml_data_call_ip_family_e ip_family out: ml_data_call_info_s *info out: ml_data_call_error_e *err
<pre>int ML_SetDataDefaultSim (int32_t slotId)</pre>	Set the main data card slot id. Before set the main data card slot, for example, set the main data slot from slot0 to slot1, you should deactivate all the pdn connections, and the call ML_SetDataDefaultSim(1), then create the pdn connections on slot1
<pre>int ML_GetDataDefaultSim ()</pre>	Get the main data card slot.

1.2.2.5 Call Interface and Call Sequence Description

All call module interfaces need to be called based on network registration.

Table 1-6. Call module interface description

Interface/Struct	Description
typedef struct { int32_t CallId; char PhoneNum[32]; E_ML_VCALL_STATE_TYPE_T State; int32_t call_end_reason; } ML_VCALL_INFO_T;	Struct of call message information. CallId: Current call index; PhoneNum[32]: The opponent's phone number; State: Call state; call_end_reason: The reason of call end;
typedef void (*ML_VCALL_MSGCB_T) (ML_VCALL_INFO_T *pvsMsg);	Call state callback function.
Int32_t ML_VcallInit(ML_VCALL_MSGCB_T cb_func)	Register the call state callback function. The callback function is triggered when a call comes in, and the call information is returned through the callback function. Parameters: out: ML_VCALL_MSGCB_T cb_func;
Int ML_VcallStart(const char *PhoneNumber)	Dial a call. Parameters: in: const char *PhoneNumber;
Int ML_VcallAnswer(void)	Answer the call. You can only answer the call when the call is incoming. You cannot answer the call in other states and will return an error. The function return value is int type. On success, 0 is returned. On error, -1 is returned.
Int ML_VcallEnd (void)	Hang up the phone. You can only hang up the phone when you are in a call state. You cannot hang up the phone in other states and will return an error. The function return value is int type. On success, 0 is returned. On error, -1 is returned.
int32_t ML_VcallCancel (void)	Cancel call callback function The function return value is int type. On success, 0 is returned. On error, -1 is returned.
int32_t ML_VcallCliStart (const char * PhoneNumber, int32_t cli)	Broadcast the call and set whether the other end is displayed. Parameters: Input: const char *PhoneNumber; int32_t cli 0: default 1: Disable display 2: Enable display The function return value is int type. On success, 0 is returned. On error, -1 is returned.
int32_t ML_VcallControl (int32_t controlType, int32_t call_id)	Suspend and resume a call. Parameters: Input: int32_t controlType 0: Suspend a call 1: Resume a call

Interface/Struct	Description
	int32_t call_id The function return value is int type. On success, 0 is returned. On error, -1 is returned.
void ML_SetAutoAnswerMode (bool onoff)	Set up automatic answering. Parameters: Input: bool onoff
int32_t ML_VcallDtmfStart (const char * Number)	dtmf dialing starts Parameters: Input: const char * Number dialing number The function return value is int type. On success, 0 is returned. On error, -1 is returned.
int32_t ML_VcallDtmfStop (void)	dtmf dialing ends Note that this API needs to be used with VcallDtmfStart. Press Stop after Start to end The function return value is int type. On success, 0 is returned. On error, -1 is returned.
int32_t ML_VcallGetCallWaiting (int32_t serviceClass, bool * enable, int32_t * serviceClassReturn)	Query the call waiting status. Parameters: 1: Voice 2: Data (referring to all bearer services) 8: Short Message Service (SMS) 16: Data Circuit Sync 32: Data Circuit Async 64: Dedicated Packet Access 128: Dedicated PAD Access Input: int32_t serviceClass specifies the service class Output: bool * enable int32_t * serviceClassReturn
int32_t ML_VcallSetCallWaiting (bool enable, int32_t serviceClass)	Set call waiting status. Parameters: Input: int32_t serviceClass specifies the service class bool * enable
int32_t ML_VcallStopCallId (int32_t call_id)	Hang up the phone with the specified id. Parameters: Input: int32_t call_id

1.2.2.6 SMS Interface and Call Sequence Description

The SMS interfaces must be called based on the network registration.

Table 1-7. Call interface description

Interface/Struct	Description
<pre>typedef enum { E_ML_SMS_FORMAT_GSM_7BIT = 0, E_ML_SMS_FORMAT_BINARY_DATA = 1, E_ML_SMS_FORMAT_UCS2 = 2, E_ML_SMS_FORMAT_IRA = 3, }E_ML_SMS_FORMAT_T;</pre>	Enum of SMS format type.
<pre>typedef struct { E_ML_SMS_FORMAT_T format; char PhoneNum[ML_SMS_MAX_ADDR_LENGTH]; int32_t SmsDataLen; char SmsData[ML_SMS_MAX_MT_MSG_LENGTH]; } ML_SMS_INFO_T;</pre>	Struct of SMS information.
<pre>typedef void (*ML_SMS_RXMSGCB_T) (ML_SMS_INFO_T *pvsMsg);</pre>	SMS callback function.
<pre>Int ML_SmsInit(ML_SMS_RXMSGCB_T cb_func)</pre>	Register SMS callback function. The callback function is triggered when receiving SMS, and the SMS information is returned through the callback function. Parameters: out: ML_SMS_RXMSGCB_T cb_func;
<pre>Int32_t ML_Sms_Sent(const ML_SMS_INFO_T *pvsSms)</pre>	Send SMS. This is asynchronous interface. After sending SMS request, it will asynchronously wait for md to return response, and will not block the thread. Parameters: in: const ML_SMS_INFO_T *pvsSms;
<pre>int32_t ML_SmsInitWithSlot (int32_t slot_id, ML_SMS_RXMSGCB_T cb_func)</pre>	Initialization SMS module with slot id, and callback function registered. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
<pre>int32_t ML_SmsInitNoDecoder (int32_t slot_id, ML_SMS_IND_T cb_func)</pre>	Indicates when new SMS is received. default is disable. enable this function , please open: meta/meta-mediatek-mtxxxxx/conf/machine/autoxxxx.conf then modify MTK_SMS_CODEC_SUPPORT = "no" => MTK_SMS_CODEC_SUPPORT = "yes". Function return value is int type. On success, 0 is returned. On error, -1 is returned.
<pre>int sendSmsNoEncoderAsync (int slot_id, char * smscPdu, char * pdu)</pre>	Send an SMS message. Based on the returned error, caller decides to resend if sending sms fails. RadioError:SMS_SEND_FAIL_RETRY means retry (i.e.

Interface/Struct	Description
	error cause is 332) and RadioError:GENERIC_FAILURE means no retry (i.e. error cause is 500) default is disable. enable this function, please open: meta/meta-mediatek-mtxxxxx/conf/machine/autoxxxx.conf then modify MTK_SMS_CODEC_SUPPORT = "no" => MTK_SMS_CODEC_SUPPORT = "yes". Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_sendSmsNoEncoderResponseCb (int32_t slot_id, ml_send_sms_no_encoder_async_response response)	Response function of sendSmsNoEncoderAsync

1.2.2.7 IMS Interface and Call Sequence Description

Table 1-8. IMS interface description

Interface/Struct	Description
Int ML_EnableIms(uint8_t on_off)	Enable/disable the IMS registration. Input parameter: On_off is 1, which means the IMS function is enabled. On_off is 0, which means the IMS function is disabled.
Int ML_GetImsRegState(int8_t *reg_state)	Get the IMS registration status. Output parameter: Reg_state value is 1, which means IMS is registered. Reg_state value is 0, which means IMS is not registered.
int ML_EnableVolte (uint8_t on_off)	Enable ims for volte. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int ML_EnableVonn (uint8_t on_off)	Enable ims for vonn, if you need to disable volte and vonn, please use ML_EnableIms(0). This API only supports slot0. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int32_t setVoNrEnabled (int32_t slot_id, bool enable)	Set Voice NR enable state. Function return value is int type. On success, 0 is returned. On error, -1 is returned.
int32_t isVoNrEnabled (int32_t slot_id, bool * enable)	Query current Voice NR enable state. Function return value is int type. On success, 0 is returned. On error, -1 is returned.

1.2.2.8 Ecall Interface and Call Sequence Description

Table 1-9. eCall control interface description

Interface/Struct	Description
<pre>typedef struct { int32_t call_id; uint32_t length; unsigned char msd_data[ML_ECALL_MSD_MAX_LENGTH]; }ml_ecall_set_msd;</pre>	Struct of MSD information.
<pre>typedef struct { int32_t arg_num; int32_t type; char address[128]; }ml_ecall_set_num;</pre>	Struct of test number/reconfiguration number information.
<pre>typedef enum { ML_EMER_CAT_MANUAL_ECALL = 1, ML_EMER_CAT_AUTO_ECALL = 2, }ml_ecall_category;</pre>	Enum of ecall category.
<pre>typedef enum { ML_ECALL_TEST = 1, ML_ECALL_EMERGENCY = 2, ML_ECALL_RECONFIG = 3, }ml_ecall_variant;</pre>	Enum of ecall variant.
<pre>typedef enum{ ML_DOMAIN_AUTO = 0, /* Automatic mode - LTE(IMS), WG(CS), 1x(C2K) */ ML_DOMAIN_CS_ONLY = 1, /* CS domain only - WG(CS) */ ML_DOMAIN_3GPP_ONLY = 2, /* 3GPP only - LTE(IMS), WG(CS) */ ML_DOMAIN_3GPP2 = 3, /* 3GPP2 only - 1x(C2K) */ ML_DOMAIN_IMS_1xCS = 4, /* IMS and 1x CS only - LTE(IMS), 1x(C2K) */ ML_DOMAIN_CS_1x = 5, /* WG CS and 1x CS only - WG(CS), 1x(C2K) */ ML_DOMAIN_IMS_ONLY = 6, /* only IMS call allowed */ }ml_ecall_domain;</pre>	Enum of radio domain when make ecall.
<pre>typedef struct{ ml_ecall_category ecall_cat; ml_ecall_variant ecall_variant; char address[20];</pre>	Struct of input parameter when requesting to make an ecall.

Interface/Struct	Description
<pre>uint32_t length; unsigned char msd_data[ML_ECALL_MSD_MAX_LENGTH]; ml_ecall_domain domain; }ml_ecall_req_msg;</pre>	
<pre>typedef struct{ int32_t data1; int32_t data2; int32_t data3; int32_t data4; }ml_ecall_pri;</pre>	Ecall priority parameter structure. data1>data2>data3>data4. The input parameters should be 1 2 3 4, which means: 1 - eCall URI set by the customer; 2 - eCall URI saved by USIM; 3 - eCall number set by the customer; 4 - eCall number saved by USIM;
<pre>typedef enum{ E_ML_ECALL_SENDING_START = 1, E_ML_ECALL_SENDING_MSD = 2, E_ML_ECALL_LACK_RECEIVED = 3, E_ML_ECALL_ALACK_POSITIVE_RECEIVED = 4, E_ML_ECALL_ALACK_CLEARDOWN_RECEIVED = 5, E_ML_ECALL_DIALING = 9, E_ML_ECALL_ALERTING = 10, E_ML_ECALL_ACTIVE = 11, E_ML_ECALL_DISCONNECTED = 12, E_ML_ECALL_IMS_ACTIVE = 13, E_ML_ECALL_IMS_DISCONNECTED = 14, E_ML_ECALL_ABNORMAL_HANGUP=15, E_ML_ECALL_IMS_MSD_ACK = 20, E_ML_ECALL_IMS_UPDATE_MSD = 21, E_ML_ECALL_IMS_IN_BAND_TRANSFER = 22, E_ML_ECALL_IMS_MSD_NACK = 23, E_ML_ECALL_IMS_SRVCC = 24, E_ML_ECALL_ONLY_DEREGISTRATION = 31, E_ML_ECALL_MAY_DEREGISTER = 32, E_ML_ECALL_PSAP_CALLBACK_START = 40, E_ML_ECALL_PSAP_CALLBACK_IMS_UPDATE_MSD = 41, E_ML_ECALL_T2_TIMEOUT = 52, E_ML_ECALL_T5_TIMEOUT = 55, E_ML_ECALL_T6_TIMEOUT = 56, E_ML_ECALL_T7_TIMEOUT = 57, E_ML_ECALL_UNSPECIFIED = 0xffff, }ML_ECall_Indication;</pre>	Enum of ecall indication type.
<pre>typedef struct{ ML_ECall_Indication ind; int call_id;</pre>	Struct of ecall indication.

Interface/Struct	Description
} ML_ECALL_IND_T;	
typedef void (*ML_ECALL_MSGCB_T)(ML_ECALL_IND_T *pvsMsg);	Callback function for reporting ecall indication.
int32_t ML_EcallIndicationInit(ML_ECALL_MSGCB_T cb_func);	Register the ecall indication callback function, and call back the cb_func function when the ecall status changes. Parameters: In: ML_ECALL_MSGCB_T cb_func
int32_t ML_ResetIvs(void);	Reset ecall state in modem and reconnect audio channel.
int32_t ML_SetMSD(ml_ecall_set_msd* msd);	Set MSD. Parameters: In: ml_ecall_set_msd* msd
int32_t ML_SetTestNumber(ml_ecall_set_num* test_num);	Set test number or URI. Parameters: In: ml_ecall_set_num* test_num
int32_t ML_SetReconfNumber(ml_ecall_set_num* reconf_num);	Set reconfig number or URI. Parameters: In: ml_ecall_set_num* test_num
int32_t ML_MakeFastEcall(ml_ecall_req_msg* msg);	Trigger an ecall and transmit MSD data in one command. Parameters: In: ml_ecall_req_msg* msg
int32_t ML_SetEmsdpri(ml_ecall_pri* pri);	Set priority of test/reconfiguration eCall number/URI The default priority is “1>3>2>4” . Parameters: In: ml_ecall_pri* pri

1.2.2.9 Suspend of Modem Interface and Call Sequence Description

Table 1-10. AT blacklist interface description

Interface/Struct	Description
Int ML_SendAT(const char* atCmd, char* finalRsp, uint32_t resp_len, int64_t timeout_ms); Int ML_SetUnsolicitedResponseFilter(ML_UnsolicitedResponseFilter filter)	Before entering IPO, enable the AT URC blacklist filtering function: 1. Call ML_SendAT function to send AT command: AT+EURCFLT=1. 2. Call ML_SetUnsolicitedResponseFilter(0) After exiting IPO, disable the AT URC blacklist filtering function: 1. Call ML_SendAT function to send AT command: AT+EURCFLT=0 2. Call ML_SetUnsolicitedResponseFilter(0xFF)

Table 1-11. MIPC keep alive interface description

Interface/Struct	Description
<pre> typedef struct { ML_keealiveType type; char sourceAddress[MAX_INADDR_LEN]; int32_t sourcePort; char desinationAddress[MAX_INADDR_LEN]; int32_t desinationPort; int32_t maxKeepAliveIntervalMillis; int32_t profile_idx; } ML_keealiveRequest typedef struct { KEEPALIVE_ACTIVE = 0, KEEPALIVE_INACTIVE, KEEPALIVE_PENDING } ML_keealiveStatusCode typedef struct { int32_t sessionHandle; ML_KeepaliveStatusCode code; } ML_keealiveStatusCode int ML_StartKeepAlive (ML_KeepaliveRequest req, ML_KeepaliveStatus * ml_status) int ML_StopKeepAlive (uint32_t sessionHandle) int ML_SetKeepAliveStatusInd (ml_keealive_cb_t evt_cb) </pre>	To keep Internet PDN/PDU session in connected mode by sending a dummy packet without waking up AP processor. Flow: 1. AP calls ML_StartKeepalive. The callback keealiveStatus status is pending, indicating that it is waiting for the AP to send the first UL packet. 2. After the MD receives the first UL packet sent by the AP, the callback keealiveStatus status is active, indicating that the MD can send keep-alive packets. 3. If an exception occurs, the callback keealiveStatus status is inactive when the modem finally fails to receive the network keep-alive response.

1.3 Frequently Asked Questions/Troubleshooting

1.3.1 SIM/CALL/SMS/Telephony Network Tips

1. How to turn off/on a specific network type, refer to "[FAQ17447] [NW] Turn off a specific network type".
2. SIM test failure pre-debug:
 - (1) The initial value of IMEI is empty and needs to be burned before it can be obtained.
 - (2) If SIM recognition is abnormal, capture the test scene mtklog including the boot process to assist in further analysis.
3. Network test failure pre-debug:
 - (1) Confirm that the platform has burned RF/IMEI, the antenna is correctly installed, and the SIM card is correctly inserted and has no outstanding fees.
 - (2) If (1) still fails to attach network after confirmation, capture the test scene mtklog including the boot process to assist in further analysis.
4. CALL/SMS test failure pre-debug:

- (1) Use the following command to confirm whether the platform has successfully attached network. If not, solve the network problem first.
`adb shell /usr/bin/mlclient_test --gtest_filter=MLClientTest.ML_GetNetState`
- (2) If (1) has been successfully attached to the network. Capture the test scene mtklog to assist in further analysis.

1.3.2 Data Tips

Modification of APN configuration file, refer to “[FAQ21414] MTK apns-conf.xml configuration guide”.

The configuration file used by T-Box is named `apns-conf.xml`, code path: `src/telephonyware/3.1/libvendor-ril/apn/resource/apns-conf.xml`, install path in the platform `/system/etc/tele/apns-conf.xml`.

If you modify `/system/etc/tele/apns-conf.xml` directly, you need to delete `apn.db` and restart to take effect.
`apn.db` path: `/data/vendor/telephony/apn.db`

1.3.3 Network Tips

1. Checking for successful network establishment:
 - 1) The network interface is up and the IP address exists, confirmed by the `ifconfig` command.
 - 2) Network IP rule & IP route & DNS settings, confirmed by the `IP rule/IP route/dumpsys netd/dumpsys dnsresolver` commands.
2. Checking for network connectivity:
Manually ping a certain IP or URL to see if it can be pinged.

1.3.4 IMS Tips

1. How to enable or disable IMS automatically when the device is powered on by default?
 - 1) The current system automatically enables IMS when the device is powered on by default.
 - 2) You can change whether IMS is supported by changing `MTK_VOLTE_SUPPORT` to yes or no in `ProjectConfig.mk`.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.